

To show a session expiry dialog with `fetch()`, create a wrapper function (or use the `fetch-intercept` library) to intercept responses, check for a `401 Unauthorized` status, and trigger a modal. The wrapper intercepts responses, parses the status, and acts before returning data to the application.  Stack Overflow +3

Implementing Fetch Interceptor for Session Expiry

Since `fetch` does not have built-in interceptors like Axios, you must override the global fetch function. 

1. Create the Custom Fetch Wrapper

javascript

```
const { fetch: originalFetch } = window;

window.fetch = async (...args) => {
  let [resource, config] = args;

  // 1. Perform request
  const response = await originalFetch(resource, config);

  // 2. Intercept response
  if (response.status === 401) {
    // Session expired, trigger modal
    showSessionExpiredDialog();
    // Optionally return a rejection
    return Promise.reject(new Error('Session Expired'));
  }

  return response;
};

// Function to trigger UI modal
function showSessionExpiredDialog() {
  console.log("Session expired. Redirecting to login...");
  // document.getElementById('session-modal').style.display = 'block';
  // window.location.href = '/login';
}
```

Key Considerations

- **401 Errors:** The `401 Unauthorized` status is the standard signal that a token or session has expired.

- **Modal Logic:** The `showSessionExpiredDialog()` function should display a modal and potentially redirect the user to the login page.
- **Using Libraries:** Alternatively, you can use the `fetch-intercept` library to simplify the process:

bash

```
npm install fetch-intercept
```

javascript

```
import fetchIntercept from 'fetch-intercept';

fetchIntercept.register({
  response: function (response) {
    if (response.status === 401) {
      showSessionExpiredDialog();
    }
    return response;
  }
});
```

- **Handling Multiple Requests:** If multiple APIs call simultaneously, you may need a flag to prevent multiple dialogs from appearing at once.  Medium +4

This pattern ensures that every `fetch` request passes through the handler, providing a centralized place to manage session expiration.